

RaspyJack — Complete Hardware Guide

Software: RaspyJack · **Upstream:** [7h30th3r0n3/Raspyjack](#) (MIT) **Platform:** Raspberry Pi (Zero 2 WH primary; Pi 3B/4B/5 alternates) + Waveshare SPI LCD HAT · **Cyber Controller profile:** raspyjack (**sd backend** — SD-card [.img](#), not esptool serial flashing) **This guide:** purchase → build → write SD image → integrate into Cyber Controller → use → troubleshoot.

1. Overview

RaspyJack is a portable offensive network "drop box" for the Raspberry Pi — Bash Bunny / LAN-Turtle / SharkJack in spirit — driven entirely from a tiny Waveshare LCD HAT and joystick, with no monitor or keyboard needed. You plug it into a target LAN by Ethernet, it boots into an on-device menu, and you run recon and attack payloads from the screen (or remotely from a browser WebUI). Upstream advertises a large payload library (~231 payloads across 13 categories per the project README — *verify against the release you install*): ARP MITM and packet sniffing, DNS spoofing, **Responder** credential capture, **nmap** scans, reverse shells / C2, Wi-Fi handshake capture and cracking, captive-portal phishing, Bluetooth tooling, and many exfiltration transports (HTTP/DNS/Discord/SMB/FTP/USB).

Cyber Controller's role here is **provisioning, not serial control**: its `sd` backend writes the SD-card image to a *removable* drive (the same hardened, removable-only writer used for Pwnagotchi and Kali ARM), then you move the card to the Pi and boot. There is no live serial protocol — the profile's `protocol` is `null` and you operate the device from its own LCD menu and WebUI, not from Cyber Controller's Devices/terminal tabs. (§9)

2. Legal & Safety

Authorized testing only. RaspyJack is a real attack platform: ARP MITM, DNS spoofing, Responder, deauth/evil-twin (with an external adapter), and captive-portal credential capture are **illegal against networks, devices, or people you do not own or lack written permission to test** (US: CFAA; plus FCC rules on RF transmit; EU and most other jurisdictions have equivalents). Because this is a *drop box* you physically plant on a network, you also need authorization for the physical access itself. Upstream states the project is "for redteam and educational purposes only" and that the user assumes full responsibility for legal compliance — treat that as binding. Run it only on your own lab gear or under an explicit, written engagement scope, and prefer passive recon first. (§9)

3. Hardware & Purchasing

RaspyJack is built from off-the-shelf Raspberry Pi parts. Pick a Pi, a matching Waveshare LCD HAT, and (for Wi-Fi attacks) an external USB adapter.

Part	Recommended	Notes	Where to buy (search terms)
Pi (primary)	Raspberry Pi Zero 2 WH (quad-core 1 GHz, 512 MB, pre-soldered header)	Upstream's main target — smallest drop-box form factor	Search "Raspberry Pi Zero 2 WH" — raspberrypi.com approved resellers (PiShop, Pimoroni, Adafruit, The Pi Hut)
Pi (alternates)	Pi 3 Model B, Pi 4 Model B (4 GB) , Pi 5 (8 GB)	Pi 4/5 support is untested upstream and the display may need adjustment — <i>verify before relying on it</i>	Same approved resellers; "Raspberry Pi 4 Model B 4GB" / "Raspberry Pi 5 8GB"
LCD HAT (option A)	Waveshare 1.44" LCD HAT — ST7735S, 128×128 (config ST7735_128)	The "original"/default; joystick + KEY1/2/3	waveshare.com or Amazon: "Waveshare 1.44inch LCD HAT"
LCD HAT (option B)	Waveshare 1.3" LCD HAT — ST7789, 240×240 (config ST7789_240)	Higher-density UI; same buttons	Waveshare / Amazon: "Waveshare 1.3inch LCD HAT"
All-in-one (option C)	M5Stack Cardputer Zero (320×170)	Upstream also supports this integrated unit (keyboard + screen) — <i>verify current support level</i>	m5stack.com / Amazon: "M5Stack Cardputer"
Ethernet for Pi Zero	Waveshare Ethernet/USB HUB HAT (3× USB + 1× RJ45)	Pi Zero has no RJ45; this HAT adds wired LAN + USB ports for adapters	Waveshare / Amazon: "Waveshare Ethernet USB HUB HAT" (chipset: <i>verify</i> — typically RTL8152B)
External Wi-Fi (attacks)	RTL8812AU : Alfa AWUS036ACH, Panda PAU09 · AR9271 : TP-Link TL-WN722N v1	Required for monitor-mode/injection — the onboard chip cannot do it (\$4)	Amazon: "Alfa AWUS036ACH", "TL-WN722N v1" (the v1 matters)

microSD	16–32 GB, Class 10 / U1, A1 (or A2)	Pi OS Lite + RaspyJack is small; size/endurance is <i>not specified upstream</i> — these are general Pi recommendations, verify	Any reputable brand (SanDisk/Samsung): "32GB A1 microSD"
Power	Zero 2 W: 5 V / 2.5 A micro-USB · Pi 4: 5 V / 3 A USB-C · Pi 5: 5 V / 5 A USB-C PD	Under-powering causes brownouts/SD corruption; <i>exact draw with HAT+adapter not specified upstream</i>	Official Raspberry Pi PSU for your model

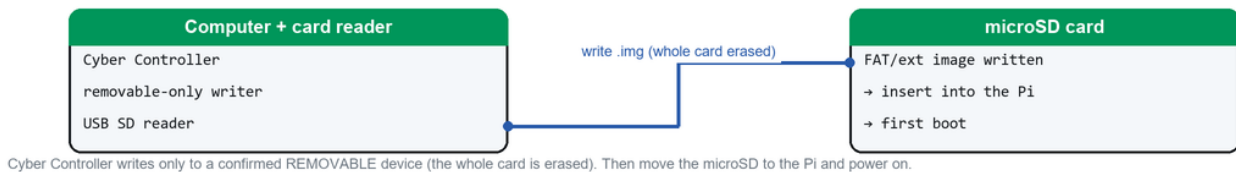
Why the onboard Wi-Fi can't attack: the Pi's built-in Broadcom (43430-class) Wi-Fi chipset does **not support monitor mode / packet injection**, so all Wi-Fi attack payloads require one of the external USB adapters above. Ethernet is what you use for the actual drop. (\$9)

4. Building / Assembly

- **Stack the HAT:** seat the Waveshare LCD HAT onto the Pi's 40-pin GPIO header (Zero 2 **WH** ships with the header pre-soldered; a plain Zero 2 W needs a header soldered/hammered on first). No wiring is required — the HAT uses the standard SPI pins.
- **SPI / GPIO pin map (BCM)** — for reference and DIY wiring (the installer enables SPI and the HAT uses these by default; *verify against your exact HAT revision*):
- **Display SPI:** MOSI=10, SCLK=11, LCD_CS=8, LCD_DC=25, LCD_RST=27, LCD_BL=24
- **Joystick:** Up=6, Down=19, Left=5, Right=26, Press/OK=13
- **Buttons:** KEY1=21, KEY2=20, KEY3=16
- **Pick the right display config:** the installer asks which screen you have — choose ST7735_128 (1.44”) or ST7789_240 (1.3”). The wrong choice = garbled or blank screen. You can change it later from the LCD menu (*Payload* → *Utilities* → *display selector*, per upstream).
- **Add networking:** plug Ethernet into the Pi's RJ45 (Pi 3/4/5) or via the Waveshare Ethernet/USB HUB HAT (Pi Zero). Plug an external Wi-Fi adapter into a USB port only if you'll run Wi-Fi payloads.
- **No special host drivers:** unlike ESP32 boards there is **no USB-serial driver / COM port** — you provision the microSD on your PC and the Pi boots from it.

5. Writing the SD image via Cyber Controller

Writing the image — Raspberry Pi / SBC (microSD)



Cyber Controller writes the image to a removable microSD/USB; then boot the device.

RaspyJack's backend is `sd` (an SD-card image written block-level to a *removable* drive), **not** esptool serial flashing. Cyber Controller's SD / Software-OS writer drives this through `sd_backend.flash_sd(profile_id, asset, device, confirmed=True)` with hard safety invariants: **removable drive only, < 256 GB, and confirmed=True** (plus Administrator/root for raw-disk access). The pipeline is: discover release asset → download (HTTPS, GitHub-allowlisted) → stream-decompress (`.xz/.gz/.zip` → `.img`, never loading the whole image into RAM) → block-write → **SHA-256 read-back verify**. (§9)

Important distribution note (read before you start). Upstream RaspyJack is currently shipped as an **installer script on top of Raspberry Pi OS Lite — the latest release (v1.0.6 at time of writing) ships no .img asset**. The raspyjack profile is wired to auto-discover a release image matching `*.img / *.img.xz / *.img.gz / *.zip` on `7h30th3r0n3/Raspyjack`, so **if/when upstream publishes a prebuilt .img, Cyber Controller writes it directly** — but until then there is nothing for it to fetch. *Verify the latest release's assets before assuming a one-click image exists.* Two practical paths:

Path A — write base Raspberry Pi OS Lite, then run the installer (works today). 1. Insert the microSD via a USB card reader (so it appears as a **removable** drive). 2. In Cyber Controller, open the **SD / Software-OS writer**. Select the target microSD in **Target (removable)** — only removable drives are listed, and **the entire drive is erased**. 3. Write a base **Raspberry Pi OS Lite (32-bit for the Zero 2 W)** image using the writer's **"Use local image..."** option (point it at a Pi OS `.img/.img.xz` you downloaded), or use **Raspberry Pi Imager** directly if you prefer its OS picker. Either way the same removable-only, confirmed write applies. 4. Run Cyber Controller as **Administrator (Windows) / root (Linux/macOS)** — raw disk writes require it. Confirm the destructive-write prompt; the writer downloads (if needed), decompresses, writes, and SHA-256-verifies the card. 5. Continue to §6 to run the RaspyJack installer on first boot.

Path B — prebuilt RaspyJack .img (only once upstream publishes one). 1. Open the SD writer and choose the **RaspyJack** profile (`raspyjack`). 2. Cyber Controller queries the latest GitHub release and lists matching `.img*` assets. If the list is empty, no prebuilt image exists yet — use **Path A**. 3. Select the asset and the removable microSD, confirm, and let the pipeline download → decompress → write → verify.

Note: the profile's `default_baud` (115200) and `protocol` (null) fields are inherited boilerplate — SD imaging does **not** use a serial baud rate or a live protocol. (§9)

6. Integrate into Cyber Controller (SD / Software-OS)

- **Profile:** `raspyjack` — backend: `"sd"`, protocol: `null`, board `"Raspberry Pi (with LCD/GPIO HAT)"`. It maps to `sd_backend.PI_IMAGE_PROFILES["raspyjack"]` (repo `7h30th3r0n3/Raspyjack`, file pattern `*.(img|img.xz|img.gz|zip)). ($9)`
- **It is a provisioning target, not a controlled device.** The standard firmware **Flash / Devices** / terminal flow is for serial boards; `flash_engine._flash_sd` deliberately does nothing on its own and defers to the removable-target SD flow. After you image the card, RaspyJack runs **standalone** — you drive it from the LCD menu and WebUI, **not** from Cyber Controller's terminal.
- **Removable-only guard (why a drive may not appear):** the writer enumerates removable / USB-bus drives (`detect_sd_cards`) and **refuses** fixed/system disks, anything ≥ 256 GB, or any device not in the freshly re-scanned removable list. If your microSD doesn't show, use a USB reader and click *Refresh drives*. This is the safety mechanism that prevents nuking your system disk.
- **Network hardening:** downloads are restricted to an HTTPS allowlist (`github.com`, `api.github.com`, `*.githubusercontent.com`, `kali.download`); other hosts are rejected. A self-built or third-party RaspyJack image therefore must be supplied via **"Use local image..."**, not an arbitrary URL.
- **Verify after write:** keep the SHA-256 read-back verify enabled — it catches a bad card or a truncated download before you ever boot the Pi.

7. Usage (end-to-end)

1. **First boot + install (Path A).** Flash Raspberry Pi OS Lite (set username/password and **enable SSH** in Imager's settings), boot the Pi, and SSH in. Then run the upstream installer (commands per the wiki — *verify against current README*):
`bash sudo apt update && sudo apt install -y git sudo su cd /root git clone https://github.com/7h30th3r0n3/Raspyjack.git mv Raspyjack Raspyjack # installer expects the folder at /root/Raspyjack cd Raspyjack chmod +x install_raspyjack.sh sudo ./install_raspyjack.sh # prompts for your LCD type, enables SPI, checks /dev/spidev0.0 sudo reboot`
The installer asks which screen you have (ST7735_128 vs ST7789_240), enables SPI, and verifies the LCD device node. After reboot the Pi comes up straight into the RaspyJack menu.
2. **Navigate the LCD menu.** Joystick **Up/Down** to move, **Left** = back, **Right / OK** = select, **KEY3** = exit (per upstream keymap). Menus offer List / Grid / Carousel views.
3. **Recon first.** From the menu run **Network Info** and an **Nmap Scan** of the target subnet (passive / least-intrusive first, authorized scope only).
4. **Run a payload.** Pick from the payload categories — e.g. **MITM & Sniff** (ARP), **DNS Spoofing**, **Responder** (credential/hash capture), captive-portal templates, or a **reverse shell**. Wi-Fi payloads (deauth/evil-twin/handshake) require the external adapter from §3.
5. **Remote control via WebUI.** RaspyJack exposes a browser UI for remote operation/editing — per upstream, main UI at `https://<device-ip>/`, HTTP fallback `http://<device-ip>:8080`, and a payload IDE at `https://<device-ip>/ide`. *Verify the exact scheme/port for your release.*
6. **Exfil / reporting.** Captures and loot can be pulled via the WebUI "Read Files" view or pushed through configured transports (e.g. Discord webhook) — set these up in the menu before deployment.

8. Troubleshooting

- **Cyber Controller won't write / "device not found in removable drives"**: use a USB card reader so the microSD is enumerated as removable, click *Refresh drives*, and run the app as Administrator/root. The writer refuses fixed disks, ≥ 256 GB media, and stale (un-rescanned) targets by design.
- **No image to download for raspyjack**: expected — upstream currently ships no `.img` release asset. Use **Path A** (base Pi OS + installer) or **"Use local image..."** with a RaspyJack/Pi OS `.img`. (§5)
- **Blank or garbled LCD**: wrong display config — rerun the installer (or *Payload* → *Utilities* → *display selector*) and pick ST7735_128 for the 1.44" or ST7789_240 for the 1.3". Also confirm the HAT is fully seated and **SPI is enabled** (`/dev/spidev0.0` must exist).
- **Buttons/joystick dead**: check the BCM pin map in §4 matches your HAT revision; a mis-seated HAT or a HAT for a different display family will report wrong GPIO.
- **No network / not reachable**: Pi Zero has no onboard RJ45 — use the Waveshare Ethernet/USB HUB HAT or a USB-OTG Ethernet adapter. Confirm DHCP/link on the target LAN before expecting the WebUI.
- **Wi-Fi attacks do nothing / "monitor mode not supported"**: the onboard chipset can't inject — plug in an RTL8812AU or AR9271 (TL-WN722N v1) adapter and confirm it enumerates and enters monitor mode.
- **Random freezes / SD corruption**: almost always under-power — use the correct official PSU for the model (§3) and a known-good microSD; re-image and re-verify if the card was suspect.
- **Pi 4 / Pi 5 oddities**: support is untested upstream and the display may need adjustment — treat these as experimental and *verify* the LCD config and pinout before deploying.

9. Sources

- Upstream repo + README: <https://github.com/7h30th3r0n3/Raspyjack> (MIT; "Raspberry Pi + WAVESHARE 1.44" LCD HAT and Cardputer Zero," redteam/educational use; release v1.0.6 carried no .img asset at writing).
- Upstream wiki: <https://github.com/7h30th3r0n3/Raspyjack/wiki> (Installation, Hardware, Keymap and UI, Menu Overview, WiFi Manager, Nmap/Responder/Reverse Shell/MITM/DNS/Network Info, Payloads, Read Files, Discord Webhook, File Structure).
- DeepWiki mirror (hardware/pin map, displays, adapters): <https://deepwiki.com/7h30th3r0n3/Raspyjack>.
- Cyber Controller profile: `src/config/profiles/raspyjack.json` (backend: sd, protocol: null, board, firmware URLs) and SD backend `src/core/backends/sd_backend.py` (`PI_IMAGE_PROFILES["raspyjack"]`, removable-only / <256 GB / confirmed=True invariants, HTTPS host allowlist, download→decompress→write→SHA-256 verify); flash dispatch `src/core/flash_engine.py::_flash_sd`; SD/OS writer UI `src/ui/qt/software_tab.py` (removable-drive picker, "Use local image...").
- Hardware vendors: Raspberry Pi approved resellers (raspberrypi.com), Waveshare (waveshare.com), M5Stack (m5stack.com). **Verify current models, prices, image-release availability, WebUI ports, and microSD/ power specs at build time — these change and several values above are marked "verify."**